

LAB 8: LỖI SỬ DỤNG USEEFFECT KHI TƯƠNG TÁC API

Tình huống 1: "Quả bom" Infinite Loop (Vòng lặp vô hạn)

Ngữ cảnh: Bạn được giao nhiệm vụ viết một component lấy danh sách bài viết từ API khi trang vừa tải lên và hiển thị chúng ra màn hình.

Mã nguồn lỗi (Bug)

```
import React, { useState, useEffect } from 'react';

export default function PostList() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    // Giả lập gọi API lấy dữ liệu
    console.log("Đang gọi API lấy danh sách bài viết...");

    // Gọi API xong thì cập nhật State
    setTimeout(() => {
      setPosts(['Bài viết 1', 'Bài viết 2', 'Bài viết 3']);
    }, 500);
  }); //QUÊN MẢNG PHỤ THUỘC (Dependency Array)

  return (
    <div>
      <h3>Danh sách bài viết:</h3>
      <ul>
        {posts.map((post, index) => <li key={index}>{post}</li>)}
      </ul>
    </div>
  );
}
```

Giải thích nguyên nhân lỗi

Vì **useEffect không có mảng phụ thuộc**, nó sẽ chạy mặc định sau **mỗi lần component render**.

Chu trình hủy diệt diễn ra như sau:

1. Component render lần đầu -> useEffect chạy.
2. useEffect gọi setPosts làm state thay đổi.
3. State thay đổi ép Component phải re-render.

4. Component re-render -> useEffect lại chạy.
5. useEffect lại gọi setPosts...

Hậu quả: Trình duyệt bị treo cứng. Nếu đây là API thật, bạn đang thực hiện một cuộc tấn công DDoS vào chính server của mình với hàng nghìn request mỗi giây.

Mã nguồn chỉnh sửa (Fix)

Cách sửa: Chỉ cần thêm một mảng phụ thuộc rỗng [] vào tham số thứ hai của useEffect.

```
useEffect(() => {
  console.log("Đang gọi API lấy danh sách bài viết...");
  setTimeout(() => {
    setPosts(['Bài viết 1', 'Bài viết 2', 'Bài viết 3']);
  }, 500);
}, []); //Đã thêm []: Chỉ chạy 1 lần duy nhất sau khi Mount
```

Tình huống 2: Kê cấp RAM - Memory Leak (Rò rỉ bộ nhớ)

Ngữ cảnh: Bạn có một Component MouseTracker hiển thị tọa độ chuột của người dùng. Component này có thể được Bật/Tắt từ một Component cha.

Mã nguồn lỗi (Bug)

```
import React, { useState, useEffect } from 'react';

export default function MouseTracker() {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handleMouseMove = (e) => {
      console.log("Chuột đang di chuyển...");
      setPosition({ x: e.clientX, y: e.clientY });
    };

    // Đăng ký sự kiện lắng nghe chuột trên toàn bộ Window
    window.addEventListener('mousemove', handleMouseMove);

    //THIẾU HÀM CLEANUP (Dọn dẹp)
  }, []);

  return <h3>Tọa độ chuột: {position.x}, {position.y}</h3>;
}
```

Giải thích nguyên nhân lỗi

Khi component bị Tắt (Unmount) bởi component cha, phần tử giao diện biến mất, nhưng sự kiện `window.addEventListener` đã đăng ký trực tiếp lên hệ điều hành/trình duyệt thì **không tự động biến mất**.

- Mỗi lần bạn Bật lại component, nó lại đăng ký thêm một sự kiện mới.
- Bật/Tắt 10 lần -> Có 10 hàm `handleMouseMove` chạy ngầm song song mỗi khi bạn rê chuột.

Hậu quả: Ứng dụng ngốn RAM nghiêm trọng, giao diện giật lag mỗi khi rê chuột, Console báo lỗi *"Can't perform a React state update on an unmounted component"*.

Mã nguồn chỉnh sửa (Fix)

Cách sửa: Trả về một hàm dọn dẹp (Cleanup function) chứa `removeEventListener`.

```
useEffect(() => {
  const handleMouseMove = (e) => {
    setPosition({ x: e.clientX, y: e.clientY });
  };

  window.addEventListener('mousemove', handleMouseMove);

  // Dọn dẹp sự kiện khi component bị unmount
  return () => {
    window.removeEventListener('mousemove', handleMouseMove);
    console.log("Đã gỡ bỏ sự kiện lắng nghe chuột!");
  };
}, []);
```

Tình huống 3: Kẽ chậm chân - Race Condition (Xung đột tiến trình)

Ngữ cảnh: Một ứng dụng có 2 nút "Xem User 1" và "Xem User 2". Khi click sẽ đổi `userId` và gọi API. Nhưng server bị lỗi nên User 1 phản hồi rất chậm (mất 3 giây), còn User 2 phản hồi nhanh (mất 1 giây).

Mã nguồn lỗi (Bug)

```
import React, { useState, useEffect } from 'react';

export default function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
```

LAB 8: LỖI SỬ DỤNG USEEFFECT KHI TƯƠNG TÁC API

```
// Giả lập API call
const fetchUser = () => {
  setUser(null); // Reset giao diện sang trạng thái loading

  // Giả lập API: User 1 tốn 3s, các User khác tốn 0.5s
  const delay = userId === 1 ? 3000 : 500;

  setTimeout(() => {
    setUser({ id: userId, name: `Tên của User ${userId}` });
  }, delay);
};

fetchUser();
}, [userId]); // Chạy lại khi userId thay đổi

if (!user) return <p>Đang tải dữ liệu...</p>;
return <h3>Đang hiển thị thông tin: {user.name}</h3>;
}
```

Giải thích nguyên nhân lỗi

1. Bạn click "Xem User 1": Request số 1 bắt đầu chạy ngầm (đếm ngược 3 giây).
2. Ngay lập tức (sau 0.5s), bạn đổi ý click "Xem User 2": Request số 2 bắt đầu chạy ngầm (đếm ngược 0.5 giây).
3. Request 2 về đích trước (tổng thời gian 1s). Giao diện hiện đúng **"Tên của User 2"**.
4. 2 giây sau, Request 1 lúc này mới chạy xong. Hàm setUser của nó kích hoạt và **ghi đè** lên kết quả cũ.

Hậu quả: Dù người dùng đang ở tab của User 2, giao diện lại đột ngột nhảy sang thông tin của User 1. Dữ liệu bị sai lệch hoàn toàn.

Mã nguồn chỉnh sửa (Fix)

Cách sửa: Sử dụng biến cờ ignore để chặn việc cập nhật state từ các request đã "hết hạn".

```
useEffect(() => {
  let ignore = false; // Cờ hiệu kiểm soát tình trạng của effect

  const fetchUser = () => {
    setUser(null);

    const delay = userId === 1 ? 3000 : 500;
```

LAB 8: LỖI SỬ DỤNG USEEFFECT KHI TƯƠNG TÁC API

```
    setTimeout(() => {  
      // Chỉ cập nhật State nếu người dùng chưa chuyển sang userId khác  
      if (!ignore) {  
        setUser({ id: userId, name: `Tên của User ${userId}` });  
      }  
    }, delay);  
  };  
  
  fetchUser();  
  
  // Khi userId đổi, Cleanup function chạy, đổi cờ hiệu của request cũ thành true  
  return () => {  
    ignore = true;  
  };  
}, [userId]);
```